Übungsblatt 3: Threads, Präemption und zeitgesteuertes Warten

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Zurückkehren, warten oder weiterrechnen? (6 Punkte)

Eine Sortierstation soll alle 200 ms eine neue Messung bearbeiten.

1. Im kooperativen Programm wird `MessungStep()` aus der Hauptschleife aufgerufen. In einem RTOS-Projekt gibt es stattdessen den Thread `Messung(void *argument)`. Wo darf eine Endlosschleife stehen und wie kommt jeweils die nächste Aufgabe zum Zug? (2 Punkte)
2. Der RTOS-Thread ruft `osDelay(7)` auf. Geben Sie seine Zustandsfolge bis zum nächsten Lauf an. Welche Einheit hat die Sieben und warum garantiert das Ablaufen der Wartezeit noch keinen sofortigen Start? (2 Punkte)
3. Jemand ersetzt den Aufruf durch die übliche `HAL_Delay(7)`. Wird der Thread dadurch Blocked? Können höher priorisierte Threads und zugelassene Interrupts trotzdem ausgeführt werden? (2 Punkte)

Musterlösung: `MessungStep` muss nach endlicher Arbeit zurückkehren. Der Dispatcher ruft dann einen anderen bereiten Schritt auf. Die dauerhafte Schleife liegt im kooperativen Hauptprogramm; der Anwendungsfortschritt wird ausdrücklich gespeichert. Ein RTOS-Thread darf seine Arbeits-/Warteschritte in einer Endlosschleife wiederholen: Der Kernel sichert den Kontext mit eigenem Stack und kann höhere bereite Threads auswählen. Ohne echtes Warten kann ein hochpriorer Thread trotzdem niedrigere aushungern.

Bei `osDelay(7)` gilt Running → Blocked → Ready → Running. Sieben bezeichnet Kernelticks. Nach Ablauf muss der Scheduler erst zuteilen; höhere bereite Prioritäten können den Start verzögern.

Die übliche HAL-Funktion wartet aktiv und macht den Thread nicht durch die Kernel-Warteverwaltung Blocked. Höhere bereite Threads können ihn präemptieren und nicht gesperrte Interrupts laufen. HAL-Millisekunden und Kernelticks sind getrennte Zeitbasen. Je zwei Punkte für die begründeten Antworten.

Aufgabe 2: Drei endliche Arbeiten auf einem Kern (10 Punkte)

Für eine Paketwaage gelten feste Prioritäten und Präemption. Alle API-, Interrupt- und Wechselkosten sind null. Die Tabelle enthält Freigaben und eigene CPU-Zeiten, keine Boardmessungen.

Thread	Priorität	Freigaben / Verhalten	CPU-Zeit pro Job
Ausschleusen	höchste	wartet bis 3 ms; ein Job, danach wieder warten	2 ms
Wiegen	mittlere	periodisch bei 0, 6, 12 ms; nach jedem Job bis zur nächsten Freigabe warten	2 ms
Etikettbericht	niedrigste	ein Job bei 0; nach Abschluss warten	5 ms

Für Wiegen gilt die relative Deadline $D = 3\,\text{ms}$. Der Ausschleusethread ist bei null bereits Blocked; andere Arbeit fehlt. Leerlauf wird als Idle bezeichnet.

1. Zeichnen Sie die CPU-Belegung im halboffenen Intervall $[0, 15)\,\text{ms}$. Kennzeichnen Sie die Ursachen der Wechsel. (4 Punkte)
2. Nennen Sie die Zustände der drei Threads bei 2,5 und 3,5 ms. (2 Punkte)
3. Berechnen Sie die CPU-Zeitanteile einschließlich Idle in diesem Intervall und die Antwortzeiten aller drei Wiegejobs. (2 Punkte)
4. Ausschleusen erhält zusätzlich einen zweiten Job bei 6 ms. Wann startet und endet der ebenfalls bei 6 ms freigegebene Wiegejob? Hält er seine Deadline? (2 Punkte)

Musterlösung:

Intervall in ms	CPU	Ursache am Anfang
$[0, 2)$	Wiegen	höchste bereite Priorität
$[2, 3)$	Etikettbericht	Wiegen wartet
$[3, 5)$	Ausschleusen	Ereignis macht höchsten Thread bereit
$[5, 6)$	Etikettbericht	Ausschleusen wartet wieder
$[6, 8)$	Wiegen	periodische Freigabe
$[8, 11)$	Etikettbericht	Wiegen wartet
$[11, 12)$	Idle	Bericht fertig, alle drei warten
$[12, 14)$	Wiegen	periodische Freigabe
$[14, 15)$	Idle	Wiegen wartet

Bei 2,5 ms: Ausschleusen Blocked, Wiegen Blocked, Etikettbericht Running. Bei 3,5 ms: Ausschleusen Running, Wiegen Blocked, Etikettbericht Ready. Präemption ist kein sachliches Blockieren des Berichts.

Wiegen erhält 6 ms, Ausschleusen 2 ms, Etikettbericht 5 ms und Idle 2 ms, zusammen 15 ms. Die Zeitanteile sind entsprechend $6/15 = 40\,\%$, $2/15 \approx 13{,}33\,\%$, $5/15 \approx 33{,}33\,\%$ und $2/15 \approx 13{,}33\,\%$. Die Wiegejobs enden bei 2, 8 und 14 ms und haben jeweils $R = 2\,\text{ms} \leq D$.

Mit dem zusätzlichen Ereignis läuft Ausschleusen von 6 bis 8, Wiegen von 8 bis 10 ms. Für den Wiegejob gilt $R = 10 - 6 = 4\,\text{ms} > 3\,\text{ms}$; seine absolute Deadline 9 ms wird verfehlt. Bereitschaft bei 6 ms ist kein Start bei 6 ms.

Aufgabe 3: Ein Tickplan mit und ohne Drift (8 Punkte)

1. Ein Kernel hat $f_{\text{tick}} = 250\,\text{Hz}$. Berechnen Sie Tickperiode und aufgerundete Tickzahlen für 6 ms und 18 ms. Warum garantiert `osDelay(1)` keine physische Mindestwartezeit von einer ganzen Tickperiode? (3 Punkte)
2. Für diese Teilaufgabe gilt stattdessen ein 1-kHz-Tick. Drei Arbeitsphasen benötigen nacheinander 3, 1 und 5 ms. Der erste Start ist bei null, alle Arbeitsenden liegen genau auf Tickgrenzen; weitere Störungen fehlen. Bestimmen Sie die nächsten drei Starts bei „Arbeit, dann `osDelay(12)`“ und bei einem absoluten 12-Tick-Sollraster. (3 Punkte)
3. Eine Arbeit endet bei Tick 53; der nächste geplante Start bei Tick 48 ist bereits verpasst. Die Periode beträgt 12 Ticks. Geben Sie eine geeignete Reaktion ohne sofortige Nachholserie an. Was kann ein Zähler verpasster Warteziele aussagen, was ohne zusätzliche Deadlinedefinition nicht? (2 Punkte)

Musterlösung: $T_{\text{tick}} = 1/250\,\text{s} = 4\,\text{ms}$. Mit $n = \lceil t f_{\text{tick}}/1000 \rceil$ ergeben sich für 6 ms zwei und für 18 ms fünf Ticks. Ein Aufruf kurz vor der nächsten Tickgrenze kann bereits an dieser Grenze wieder Ready werden. Die anschließende Zuteilung kann zusätzlich verzögert sein; die Umrechnung allein garantiert keine physische Mindestwartezeit.

Relative Starts: $0 + 3 + 12 = 15$, $15 + 1 + 12 = 28$ und $28 + 5 + 12 = 45\,\text{ms}$. Absolute Starts im ungestörten Modell: 12, 24 und 36 ms. Das absolute Raster addiert die Arbeitszeit nicht auf jede Periode; es erzwingt unter Last keinen pünktlichen tatsächlichen Start.

Eine zulässige Politik zählt die Zielverfehlung, richtet das nächste Ziel auf $53 + 12 = 65$ aus und blockiert bis dorthin. Alternativ kann ein spezifizierter Entwurf veraltete Aktivierungen überspringen. Eine sofortige Serie wäre für viele Ausgänge unerwünscht und erzeugte zusätzliche Last. Der Zähler zeigt verpasste Warteziele, beweist aber ohne passenden Job-/Deadlinevertrag weder Anzahl noch Ausbleiben aller Deadline-Misses.

Aufgabe 4: Ressourcen und falsche Abhilfen (8 Punkte)

1. Drei CMSIS-Threads erhalten Stacks mit 768, 1536 und 640 Byte. Pro Thread werden zusätzlich 96 Byte Verwaltungsdaten angenommen. Berechnen Sie die Summe. Welche weiteren Anteile fehlen für eine vollständige Speicherbilanz? Wie viele Byte sind 192 native `StackType_t`-Elemente zu je vier Byte? (3 Punkte)
2. Ein Diagnose-Thread steht über einer Ausgabesteuerung, bleibt ständig Ready und ruft in seiner Rechenschleife `osThreadYield()` auf. Die Ausgabesteuerung bleibt ebenfalls Ready. Wird sie dadurch ausgewählt? Benennen Sie das Problem und eine geeignete Änderung. Es gibt keine gemeinsam gehaltene Ressource. (3 Punkte)
3. Die logische Interruptschwelle für Kernelaufrufe ist 6; vier Prioritätsbits und passende Gruppierung sind vorausgesetzt. Darf ein IRQ mit logischer Priorität 4 `osThreadFlagsSet` aufrufen? Darf ein hierfür zulässiger IRQ mit Priorität 7 `osThreadFlagsWait` aufrufen? (2 Punkte)

Musterlösung: $768 + 1536 + 640 + 3 \cdot 96 = 3232\,\text{Byte}$. Idle-/Timer-Task, weitere Objekte, Allokationszuschläge und Ausrichtung fehlen. Die TCB-Größe ist hier nur angenommen. Für den Gesamt-RAM-Bedarf sind auch Handlerstack, weitere Daten und das konkrete Speicherlayout zu beachten; im Heap enthaltene Blöcke nicht nochmals außerhalb zählen. Native Elemente: $192 \cdot 4 = 768\,\text{Byte}$. CMSIS `stack_size` ist bereits in Byte angegeben.

Yield macht den Diagnose-Thread nicht Blocked. Er bleibt der höchste bereite Thread; die Ausgabe verhungert (Starvation). Echtes Warten auf einen nächsten Termin oder ein Ereignis

kann Zuteilung ermöglichen; auch ein fachlich begründeter niedrigerer Diagnoserang ist möglich. Ohne Ressourcenabhängigkeit liegt keine Priority Inversion vor.

IRQ-Priorität 4 ist dringlicher als die erlaubte Schwelle 6 und darf in dieser Konfiguration keine Kernel-API aufrufen. Priorität 7 ist für passende nichtblockierende Kernelaufrufe erlaubt. `osThreadFlagsWait` ist jedoch ein Threadaufruf und auch aus diesem IRQ unzulässig. Der IRQ darf einen Hinweis mit `osThreadFlagsSet` übergeben, nicht selbst blockierend darauf warten.

Aufgabe 5: Wie viele Hinweise bleiben erhalten?

Ein Thread wartet auf Bit 2, also die Maske `0x04U`. Ein zulässiger anderer Thread setzt dieses Bit bei 2, 3 und 9 ms. Der Empfänger ruft `osThreadFlagsWait(0x04U, osFlagsWaitAny, osWaitForever)` bei 5 und 11 ms und direkt nach dem zweiten erfolgreichen Wait noch einmal auf. Vorher ist das Bit gelöscht, weitere Set-Aufrufe fehlen. Erfolgreiche Waits löschen das angenommene Bit automatisch.

1. Welche beiden Waits kehren sofort zurück und was passiert beim dritten?
2. Wie viele Set-Aufrufe und wie viele erfolgreiche Waits gibt es? Warum sind diese Zahlen verschieden?
3. Wie erkennen Sie einen Fehler im Rückgabewert einer Flagfunktion? Weshalb wäre ein Vergleich mit `osOK` falsch?

Musterlösung: Bei 5 ms ist das Bit durch die Set-Aufrufe bei 2 und 3 ms gesetzt. Der Wait kehrt sofort zurück und löscht es. Bei 11 ms wird das bei 9 ms gesetzte Bit angenommen und gelöscht. Der dritte Wait blockiert, da kein Hinweis mehr aussteht.

Drei Set-Aufrufe ergeben zwei erfolgreiche Waits. Wiederholtes Setzen eines bereits gesetzten Bits speichert nur einen ausstehenden Hinweis, keinen Ereigniszähler. Flagfunktionen liefern Flagbits oder einen Fehlerwert mit gesetztem `osFlagsError`-Bit. Fehlerprüfung: `(flags & osFlagsError) != 0U`. `osOK` ist der Statuscode anderer APIs und kein Test für erfolgreich gelieferte Flagbits.

Aufgabe 6: Zwei FreeRTOS-Threads mit LEDs

Programmieren, etwa 20–30 Minuten. Nutzen Sie die vorbereitete **NUCLEO-H755ZI-Q / STM32H755ZIT6**-Basis mit STM32CubeMX und VS Code mit STM32 VS Code Extension. Ausschließlich **M4** läuft, M7 bleibt deaktiviert. Die Projektvorbereitung erfolgt vor dem Termin.

Die Basis verwendet **FreeRTOS / CMSIS-RTOS2**, Präemption und 1000 Kernelticks pro Sekunde. `Lab03_Start()` legt genau diese zwei Anwendungsthreads an:

Thread	Priorität	LED und Verhalten
`Lab03_Heartbeat`	`osPriorityNormal`	LD2/PE1 umschalten, 500 Ticks blockierend warten
`Lab03_HeavyMath`	`osPriorityBelowNormal`	rechnen, nach jedem Rechenblock LD1/PB0 umschalten

Ersetzen Sie die beiden Funktionsrümpfe in `code/lab03/threads_student.c` und entfernen Sie danach `WaitForImplementation`.

1. **Heartbeat:** Rufen Sie in einer Endlosschleife `Lab03_ToggleHeartbeat()` und danach `osDelay(LAB03_HEARTBEAT_TICKS)` auf. Die Konstante ist 500. Prüfen Sie den Delay-Status; bei einem Fehler rufen Sie `Lab03_Fail()` auf. Welche vollständige Hell-Dunkel-Periode erwarten Sie ungefähr?
2. **HeavyMath:** Führen Sie in einer Endlosschleife immer wieder einen Rechenblock aus: Aktualisieren Sie `lab03_accumulator` in einer inneren Schleife `LAB03_MATH_ITERATIONS`-mal mit `lab03_accumulator * 1664525U + 1013904223U`. Die Konstante ist 10 Millionen. Rufen Sie nach jedem Block `Lab03_ToggleMath()` auf. Dieser Thread soll weder Delay noch Yield oder Interruptsperren verwenden. Die LED macht den Rechenfortschritt sichtbar; ihr Blinktempo ist nicht vorgegeben.
3. **Starten und prüfen:** Bauen und laden Sie nur das CM4-Ziel. Prüfen Sie, dass LD2 regelmäßig blinkt und LD1 nach Rechenblöcken umschaltet. Erklären Sie, warum Heartbeat trotz der dauerhaften Rechenschleife ausgeführt wird und wann HeavyMath zum Zug kommt. Geben Sie die Zustände beider Threads während der Heartbeat-Wartezeit an.

Optionaler Prioritätsversuch: Ändern Sie in `Lab03_Start()` nur die HeavyMath-Priorität auf `osPriorityAboveNormal`. Sagen Sie die Wirkung auf beide LEDs voraus und prüfen Sie sie nach einem Neustart. Stellen Sie anschließend `osPriorityBelowNormal` wieder her.

Ergebnis: zwei eigene Threadfunktionen und eine kurze Funktionsbeobachtung. Es werden ausschließlich die eingebauten LEDs verwendet. Taster, EXTI, eigene Interrupt-Handler und zusätzliche Timer entfallen. Der vorbereitete FreeRTOS-Systemtick bleibt für `osDelay` erforderlich; die HAL verwendet denselben SysTick. Ohne Boardlauf dessen Status angeben.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; Arm CMSIS-RTOS2; ST UM2408 und RM0399. Papierzeiten und Speichergrößen sind Lehrannahmen.

Musterlösung:

```
#include "lab03_rtos.h"

void Lab03_Heartbeat(void *argument)
{
    (void)argument;
    for (;;) {
        Lab03_ToggleHeartbeat();
        if (osDelay(LAB03_HEARTBEAT_TICKS) != osOK) {
            Lab03_Fail();
        }
    }
}

void Lab03_HeavyMath(void *argument)
{
    (void)argument;
    for (;;) {
        for (uint32_t i = 0U; i < LAB03_MATH_ITERATIONS; ++i) {
            lab03_accumulator = lab03_accumulator * 1664525U
                              + 1013904223U;
        }
        Lab03_ToggleMath();
    }
}
```

Heartbeat schaltet beim ersten Lauf um und blockiert danach 500 Kernelticks. Bei 1000 Hz sind das nominell 500 ms; eine vollständige Hell-Dunkel-Periode dauert ungefähr eine Sekunde. Tickphase, eigene Arbeit und verzögerte Zuteilung beeinflussen die Zeiten. `osDelay` wartet relativ zum Aufruf und erzeugt kein absolutes Sollraster.

Während Heartbeat Blocked ist, läuft HeavyMath. Nach Ablauf der Wartezeit wird Heartbeat Ready und präemptiert wegen seiner höheren Priorität HeavyMath. Dieser ist dann Ready, bis Heartbeat wieder wartet. `HAL_Delay` würde im Heartbeat aktiv warten und HeavyMath diese Rechenzeit nehmen. Idle erhält bei dauerhaft bereiter Rechenlast regulär keine CPU-Zeit.

Die Rechnung ist vorzeichenlos modulo 2^{32}. Der vorbereitete `volatile uint32_t lab03_accumulator` sorgt für beobachtbare Speicherzugriffe; nur HeavyMath schreibt ihn. `volatile` ersetzt keine Synchronisation. Das LD1-Blinktempo hängt von Takt, Übersetzung und den Unterbrechungen durch Heartbeat ab. Ist es schlecht sichtbar, kann die Blocklänge vor dem Termin angepasst werden; sie ist kein Zeitvertrag.

Beim optionalen Versuch bleibt HeavyMath mit AboveNormal ständig über dem Heartbeat. LD1 schaltet weiter um, LD2 kann schon vor der ersten Umschaltung aus bleiben. Heartbeat verhungert als Ready-Thread. Yield würde HeavyMath nicht Blocked machen und dem niedrigeren Heartbeat keine Zuteilung sichern. Die ursprüngliche Priorität wiederherstellen.

Prüfkriterien: zwei eigene Threadschleifen, blockierender Heartbeat, dauerbereite Rechenlast und je eine LED pro Thread; CM7 bleibt deaktiviert. SysTick und Kernelstart sind vorbereitet. Die LED-Beobachtung belegt Funktion, keine garantierte WCET oder Antwortzeit.